10. Tip Sistemleri ve Clean Code: AI Çıktısını Type-Safe Kılmak

'Any' Kullanımını Yasaklayarak Çalışma Zamanı Hatalarını Derleme Zamanında Yakalamak
Bölüm 2 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yapay zeka modelleri neden sürekli 'any' veya belirsiz tipler kullanmaya meyillidir ve bu tembellik projeyi nasıl çökertir?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 TypeScript veya statik tipler lüks değil; yapay zekanın saçmalamasını engelleyen en güçlü gardiyandır. - 📌 `any` kullanmak, tip denetleyicisini (type checker) kör etmek ve çalışma zamanı hatalarına davetiye çıkarmaktır. - 📌 Zod veya Pydantic gibi doğrulama kütüphaneleri, dış dünyadan gelen verinin AI kodunu patlatmasını engeller.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Havalimanı gümrük kapısını düşünün.

Gümrük memuru gelen her bavulu açıp kontrol etmek yerine: *"Geç geç, hepiniz geçin, ne olursa olsun kabulüm"* derse ne olur? Ülkeye yasaklı maddeler, kaçak eşyalar dolar. İşte TypeScript'te `any` kullanmak tam olarak budur.

Oysa sıkı bir tip sistemi (Strict Typing), bavulları x-ray cihazından geçiren memurdur: *"Bu bavulda sadece 'User' tipi kıyafetler olabilir. 'undefined' veya 'null' tespit edildi, içeri giremezsiniz!"*

Yapay zeka modelleri bazen kolaya kaçıp `any` yazar. Bir mühendisin görevi, o gümrük kapısını asla nöbetçisiz bırakmamaktır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Tip Güvenliği Zinciri:

```
[Dış Dünya: API / Form / DB]
       |
       ▼
[Çalışma Zamanı Doğrulama (Zod / Pydantic)] ──> "Girdi şemaya uyuyor mu?"
       |
       ▼
[Derleme Zamanı Güvencesi (Strict Types)] ──> "Asla undefined veya any yok"
       |
       ▼
[Saf & Temiz İş Mantığı (Clean Code)]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: TypeScript / Clean Code Gurusu
Aşağıdaki gevşek ve güvensiz yazılmış JavaScript/TypeScript fonksiyonunu incele.

## GÖREV:
1. Kodda `any` kullanımını tamamen kaldır.
2. Zod kullanarak gelen JSON verisini doğrulayan (runtime validation) bir şema yaz.
3. 'Early Return' ve Clean Code prensiplerine uygun, tip güvenli hale getir.

## KOD:
```typescript
function processOrder(data: any) {
  if (data) {
    if (data.items) {
      return data.items.map((i: any) => i.price * i.qty);
    }
  }
}
````
````

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü / Gevşek Tip: 'any' kullanımı ve tanımsız değer çökmesi riski
export function processOrderNaive(data: any): any {
  // Eğer data.items boşsa veya price string geldiyse ('100' * 2 = NaN) patlar!
  return data.items.map((item: any) => {
    return item.price * item.quantity;
  });
}
````
````

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import { z } from "zod";

// 1. Çalışma Zamanı Şeması (Runtime Schema)
export const OrderItemSchema = z.object({
  id: z.string().uuid(),
  price: z.number().positive("Fiyat pozitif bir sayı olmalıdır."),
  quantity: z.number().int().positive("Adet pozitif tam sayı olmalıdır.")
});

export const OrderSchema = z.object({
  orderId: z.string().min(1),
  items: z.array(OrderItemSchema).min(1, "En az bir sipariş kalemi bulunmalıdır.")
});

// 2. TypeScript Tipleri Otomatik Türetilir (Inference)
export type Order = z.infer<typeof OrderSchema>;
export type OrderItem = z.infer<typeof OrderItemSchema>;

export interface OrderTotalResult {
  orderId: string;
  total: number;
  itemCount: number;
}

// 3. Tip Güvenli ve Temiz Fonksiyon
export function calculateOrderTotal(rawData: unknown): OrderTotalResult {
  // Veriyi kapıda doğrula: Geçersizse anında açıklayıcı hata fırlatır
  const validated = OrderSchema.parse(rawData);

  const total = validated.items.reduce((sum, item) => {
    return sum + (item.price * item.quantity);
  }, 0);

  return {
    orderId: validated.orderId,
    total: Math.round(total * 100) / 100,
    itemCount: validated.items.length
  };
}
```

💡 **Kod Analizi:** `unknown` tipi ile dışarıdan gelen verinin belirsizliği açıkça kabul edilmiş, ardından Zod ile kapıda doğrulanmıştır. `z.infer` ile tek bir kaynaktan hem derleme hem çalışma zamanı güvenliği sağlanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Tip hatasını çözemeyince `// @ts-ignore` veya `as any` yazıp geçmek.

  - **Çözüm:** Sorunu halının altına süpürmeyin; tip daraltma (type narrowing) veya Zod kullanın.
- **Tuzak:** Sadece interface yazıp dışarıdan gelen JSON'ın buna uyduğunu varsaymak.
  - **Çözüm:** TypeScript tipleri derlendikten sonra yok olur (type erasure); runtime'da Zod veya Pydantic şarttır.
- **Tuzak:** İç içe 5 kat if-else piramitleri oluşturmak.
  - **Çözüm:** Guard Clause (Early Return) desenini uygulayarak kodu düzleştirin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 10: Tip Sistemleri ve Clean Code - Zod ile Kusursuz Doğrulama*

- 🖥️ **Slayt 1:** Neden 'any' yazılım dünyasının en tehlikeli anahtar kelimesidir?
- 🖥️ **Slayt 2:** Type Erasure nedir? Derleme zamanı vs Çalışma zamanı.
- 🖥️ **Slayt 3:** Zod ile tek kaynaktan şema ve tip türetme.
- 🖥️ **Slayt 4:** Guard Clause deseni ile iç içe if-else piramitlerinden kurtulmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir API endpoint simülasyonu yapın. Gelen bozuk JSON verisini (örneğin fiyatı string veya negatif olan bir obje) önce naive fonksiyona yollayın, `NaN` aldığını gösterin. Ardından Zod şemasından geçirin ve hatanın detaylı fırlatıldığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde `any` kullanılan en az 2 yeri bulun. `any` yerine katı bir interface veya `unknown` + Zod doğrulaması ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: TypeScript kodunda `any` yerine `unknown` kullanmanın temel farkı nedir? - A) Hiçbir farkı yoktur - B) `unknown` tipi, üzerinde işlem yapılmadan önce tip kontrolü (typeof, instanceof veya Zod) yapılmasını zorunlu kılar - C) `unknown` sadece sayılar için kullanılır - D) `unknown` kodun derlenmesini engeller > **Doğru Yanıt:** B > **Açıklama:** `any` tüm kontrolleri devre dışı bırakırken, `unknown` geliştiriciyi veriyi kullanmadan önce tipini daraltmaya zorlar.

Soru 2: Zod veya Pydantic gibi kütüphanelere neden ihtiyaç duyarız? - A) CSS yazmak için - B) TypeScript tipleri derleme sonrasında silindiği için, dışarıdan gelen verileri çalışma zamanında (runtime) doğrulamak için - C) Sadece SQL sorgularını hızlandırmak için - D) Bilgisayarın fan hızını ayarlamak için

> **Doğru Yanıt:** B > **Açıklama:** TypeScript derlendiğinde tüm tip tanımları silinir. Çalışma zamanında gelen verinin güvenliği ancak runtime validator'lar ile sağlanabilir.